

UNIVERSIDAD DE CARABOBO
FACULTAD DE CIENCIAS Y TECNOLOGÍA
ESCUELA DE COMPUTACIÓN
CAMPUS BÁRBULA
ARQUITECTURA DEL COMPUTADOR



Practica de Laboratorio 3: Entrada/Salida Mapeada en Memoria en MIPS32

Bárbula, mayo de 2026

Daniel Gutierrez CI: 32047059
Gabriel Diaz CI: 31161550

1. Organización de la memoria y el papel de las instrucciones de carga/almacenamiento

En un sistema que utiliza entrada/salida mapeada en memoria (memory-mapped I/O), no existe una separación física o lógica entre la memoria RAM y los periféricos desde la perspectiva del procesador. Todo comparte un único espacio de direcciones.

Por convención (y en arquitecturas como MIPS), los dispositivos de E/S no se mezclan con los programas de usuario. Se suelen ubicar en los extremos superiores del mapa de memoria (por ejemplo, a partir de la dirección 0xFFFF0000 en el simulador MARS), en una región reservada exclusivamente para el hardware.

La implicación directa de este diseño es que el procesador no necesita saber que está hablando con un sensor o un módulo de presión arterial; simplemente ejecuta las instrucciones estándar de lectura (lw) y escritura (sw). Cuando haces un sw hacia la dirección de control de un periférico, en realidad estás enviando un pulso eléctrico que activa el dispositivo. Cuando haces un lw en la dirección de datos, estás capturando los niveles de voltaje que el sensor ha colocado en el bus.

2. ¿Cuál es la principal diferencia entre memory-mapped I/O y la entrada/salida por puertos?

La diferencia fundamental radica en cómo el procesador ve a los periféricos:

- **Mapeada en memoria:** Periféricos y RAM comparten el mismo bus de direcciones.
- **Aislada:** Utiliza instrucciones específicas para los periféricos.

El enfoque mapeado simplifica enormemente el diseño interno de la CPU, ya que reutiliza los circuitos y registros existentes para la memoria, pero tiene la desventaja de çonsumir"direcciones que de otro modo podrían usarse para más RAM (aunque en arquitecturas de 32 o 64 bits, el espacio sobra).

El enfoque por puertos no gasta direcciones de RAM, pero requiere circuitería adicional y líneas de control específicas en la placa base.

MIPS32 utiliza E/S mapeada en memoria porque su filosofía central es RISC (Computación con Conjunto de Instrucciones Reducido). Su objetivo es mantener el hardware lo más simple y rápido posible, evitando agregar instrucciones y complejidad innecesaria a la Unidad de Control.

3. El peligro de las direcciones solapadas

En un sistema con I/O mapeado en memoria (Memory-Mapped I/O o MMIO), el procesador utiliza el mismo espacio de direcciones tanto para la memoria RAM como para los registros de los periféricos.

Si dos dispositivos usan direcciones solapadas, se produce un conflicto crítico en el bus del sistema que rompe el comportamiento correcto del hardware.

3.1. Problemas que surgen por el solapamiento

Cuando dos dispositivos comparten el mismo rango de direcciones, el sistema experimenta fallos tanto en las operaciones de lectura como en las de escritura:

- **Contención del bus (Bus Contention) en Lecturas:** Si el procesador intenta leer una dirección solapada, ambos dispositivos intentarán colocar datos en el bus de datos al mismo tiempo. Esto provoca que los transistores de un dispositivo intenten llevar una línea a nivel lógico alto (1) mientras el otro la lleva a nivel bajo (0), causando un cortocircuito momentáneo, un consumo excesivo de corriente, calentamiento y, finalmente, corrupción de datos (el procesador leerá basura).

- **Escritura duplicada o impredecible:** Si el procesador escribe en esa dirección, ambos dispositivos recibirán el mismo comando o dato simultáneamente. Esto activará funciones no deseadas en el dispositivo secundario, alterando su estado interno y provocando comportamientos erráticos.

- **Inestabilidad del sistema:** El sistema operativo experimentará fallos de página falsos, excepciones de hardware o congelamientos totales (hangs)

debido a la falta de predictibilidad en las respuestas de los periféricos.

3.2. Cómo se evita este conflicto.

El conflicto se resuelve asegurando la exclusividad mutua, es decir, garantizando que cada dirección física responda a un único componente físico. Esto se logra mediante tres enfoques combinados:

La lógica del circuito impreso (o del chipset/puente del sistema) analiza los bits del bus de direcciones para generar una señal de habilitación única llamada **Chip Select (CS)** o *Chip Enable*.

- **Decodificación completa:** Se diseñan puertas lógicas que verifican exhaustivamente todos los bits de la dirección. Si la dirección pertenece al Dispositivo A, la señal `CS_A` se activa y la señal `CS_B` se apaga obligatoriamente. El dispositivo no seleccionado mantiene sus pines de datos en **alta impedancia (Tri-state)**, desconectándose eléctricamente del bus para evitar la contención.

En arquitecturas modernas (como PCI o PCI Express), las direcciones no están fijadas en el hardware.

- Los dispositivos cuentan con **Registros de Dirección Base (BAR - Base Address Registers)**.
- Durante el arranque (POST/BIOS/UEFI), el firmware detecta cuánta memoria MMIO requiere cada dispositivo.
- El sistema operativo o la BIOS **asigna rangos de memoria contiguos y no solapados** a cada BAR. Si el Dispositivo A se mapea en `0xF0000000`, el Dispositivo B se moverá automáticamente a `0xF0001000`.

En sistemas antiguos (como el bus ISA) o en microcontroladores simples donde no existe Plug and Play:

- **Jumpers o Switches físicos:** El técnico debía configurar físicamente puentes en la tarjeta para cambiar la dirección base del dispositivo y evitar que chocara con otro.

- **Firmware/Árbol de Dispositivos (Device Tree):** En sistemas embebidos modernos (como dispositivos ARM/Linux), se define un archivo de configuración (*Device Tree*) que mapea estáticamente las direcciones de los periféricos, asegurando que el desarrollador no asigne el mismo bloque a dos controladores distintos.

4. ¿Por qué se considera que el memory-mapped I/O simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos?

El Memory-Mapped I/O (MMIO) simplifica el diseño del conjunto de instrucciones (ISA) de un procesador porque elimina por completo la necesidad de crear instrucciones dedicadas exclusivamente para Entrada/Salida (como `IN` u `OUT`).Bajo este modelo, los periféricos se ubican en el mismo mapa de direcciones que la memoria RAM. Para el procesador, un sensor o una pantalla se comportan exactamente igual que una celda de memoria convencional.Esto aporta tres grandes ventajas al diseño del procesador (especialmente en arquitecturas RISC como MIPS32):

- Cero Opcodes Nuevos: La Unidad de Control del procesador no requiere decodificadores ni circuitos lógicos adicionales para interpretar comandos específicos de hardware. Solo necesita saber procesar las instrucciones nativas de carga y almacenamiento (como `lw` y `sw`).
- Unificación de Formatos: Todas las interacciones con los periféricos se empaquetan de forma idéntica dentro de los formatos de instrucción ya existentes (como el Formato Tipo-I en MIPS). El procesador no tiene que alterar su estructura de bits para "hablarçon dispositivos externos.
- Reutilización de Modos de Direccionamiento: El único modo de direccionamiento de datos de MIPS32, que es base + desplazamiento(`offset($register)`),

sirve tanto para acceder a variables, vectores y pilas en la RAM, como para leer el registro de control o de datos de cualquier periférico.

El MMIO traslada toda la complejidad de separar los dispositivos fuera del procesador (dejándola en manos de los decodificadores de direcciones del hardware externo), lo que permite mantener el núcleo de la CPU lo más simple, compacto y rápido posible.

Si una arquitectura como MIPS32 optara por E/S por puertos (Port-Mapped I/O o PMIO), el procesador aislaría por completo el espacio de memoria RAM del espacio de dispositivos mediante una línea de control física dedicada.Para poder comunicarse con este espacio segregado, se vería obligado a incluir instrucciones adicionales e independientes en su repertorio (ISA), complicando el decodificador de la unidad de control:

- Instrucciones de Transferencia Básicas (`in` y `out`): Se necesitarían opcodes exclusivos para mover datos desde y hacia los puertos (ej. `in $t0, puerto` y `out $t0, puerto`). En MIPS32, esto requeriría crear un formato de instrucción modificado para poder codificar el número de puerto dentro de los 32 bits de la instrucción.

- Duplicación según el Tamaño del Dato: MIPS32 maneja diferentes tamaños de datos nativamente con `lb`, `lh` y `lw`. Si usara puertos, el hardware del procesador tendría que duplicar estas variantes agregando al menos seis instrucciones nuevas:

- **Para Bytes (8 bits):** `inb` y `outb`

- **Para Media Palabra (16 bits):** `inh` y `outh`

- **Para Palabras (32 bits):** `inw` y `outw`

- Instrucciones de Bloque o Cadena (`ins` y `{outs`): Para evitar la lentitud de transferir datos byte por byte en un bucle, se necesitarían instrucciones complejas de hardware que lean directamente de un puerto y escriban en una dirección de la RAM en un solo ciclo de reloj, rompiendo por completo la filosofía de simplicidad RISC.

5. ¿Qué ocurre en los buses de datos y direcciones?

A nivel físico, el procesador **no tiene ni idea** de si está hablando con la RAM o con un periférico; él simplemente ve números y pone voltajes en los cables.Esto es lo que pasa exactamente en el hardware cuando se accede a una dirección mapeada para un dispositivo (MMIO):

- **El bus de direcciones:** El procesador coloca la dirección física (por ejemplo, `0xFFFF0000`) en los cables del bus de direcciones.
- **El bus de control:** La CPU activa la señal de lectura o escritura (como `MemRead` o `MemWrite` en MIPS32) para indicar qué quiere hacer.
- **El bus de datos:** Si es una escritura (`sw`), la CPU pone los datos en los cables. Si es una lectura (`lw`), la CPU se queda esperando a que alguien ponga los datos en el bus.

5.1. ¿Cómo sabe el hardware que debe ir al periférico y no a la RAM?.

Aquí es donde entra el verdadero "truco": un circuito externo llamado **decodificador de direcciones** (que suele estar integrado en el chipset de la placa madre).Este circuito funciona como un policía de tráfico que analiza constantemente los bits que viajan por el bus de direcciones:

- **Si la dirección está en el rango de la RAM** (ej. direcciones bajas), el decodificador activa la señal de habilitación de la memoria RAM (*Chip Select*), y la RAM responde.
- **Si la dirección pertenece al rango del periférico** (ej. direcciones altas como `0xFFFF0000`), el decodificador **apaga la RAM** (la ignora por completo) y activa la señal de habilitación (*Chip Select*) de ese periférico específico (como el sensor de luz).

Al activarse, el periférico toma el control, lee los datos del bus (si era una escritura) o coloca su información en el bus de datos (si era una lectura) para que el procesador la reciba.

6. ¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria?

No directamente. En los sistemas operativos modernos, un programa normal de usuario (modo usuario o sin privilegios) no puede acceder de forma directa a las direcciones de memoria física donde están mapeados los dispositivos de hardware. Si intentara hacerlo a la fuerza, el sistema operativo abortaría el programa inmediatamente.Para evitar estos accesos no autorizados y proteger la estabilidad del sistema, existen dos mecanismos principales combinados:**Mecanismos de protección:**

- **Unidad de Gestión de Memoria (MMU) y Memoria Virtual:** Los programas de usuario no ven las direcciones físicas reales de la máquina; trabajan en un espacio de **memoria virtual**. La MMU (un componente de hardware del procesador) utiliza unas tablas de páginas gestionadas por el sistema operativo. El sistema operativo simplemente **no mapea** las direcciones físicas de los periféricos en el espacio virtual del programa normal.
- **Bits de Privilegio (User/Supervisor):** En cada página de memoria se configuran bits de permisos. El rango de direcciones del MMIO se marca estrictamente como de nivel **Supervisor / Kernel**. Si el procesador intenta ejecutar un `lw` o `sw` en esa zona estando en modo usuario, la MMU genera una excepción de hardware (un fallo de protección o *Segmentation Fault*), dándole el control al sistema operativo para que cierre el programa infractor.

7. ¿Qué técnicas se pueden emplear para evitar esperas activas innecesarias al interactuar con dispositivos?

La espera activa (*polling*) quema ciclos de reloj del procesador de forma inútil (como el bucle `espera_init` de tu ejercicio anterior). Para evitar este desperdicio, se emplean principalmente dos técnicas:**1. E/S dirigida por Interrupciones (Interrupt-Driven I/O)**En lugar de que el procesador esté preguntando constantemente *"¿Ya estás listo?"*, **el periférico es el que avisa activamente al procesador cuando termina su tarea**.

- **Cómo funciona:** El procesador inicia el sensor y se pone a hacer otras tareas útiles (o se duerme en un estado de bajo consumo). Cuando el sensor tiene el dato listo, envía una señal eléctrica a través de una línea física dedicada llamada **IRQ (Interrupt Request)**.
- **Respuesta:** El procesador detiene inmediatamente lo que estaba haciendo, guarda su estado actual y salta a ejecutar una función especial llamada **Rutina de Servicio de Interrupción (ISR)** para leer el dato. Al terminar, vuelve a lo que estaba haciendo originalmente.

Cuando el dispositivo maneja grandes volúmenes de datos (como un disco duro, una tarjeta de red o una tarjeta de sonido), usar interrupciones por cada byte satura al procesador.

- **Cómo funciona:** Se introduce un chip secundario especializado llamado **Controlador DMA**. El procesador le dice al DMA: *"Por favor, copia estos 4 Megabytes de datos desde el sensor de luz directamente a la RAM en la dirección X"*.
- **Resultado:** El DMA se encarga de mover toda la información de forma autónoma a través de los buses del sistema sin que el procesador intervenga en absoluto. Una vez que el DMA ha terminado de copiar todo el bloque, le envía **una única interrupción** al procesador para decirle: *"Listo jefe, todo el bloque de datos ya está en la RAM"*.

8. Análisis y discusión de resultados

Los dos programas desarrollados en lenguaje ensamblador **MIPS32** implementan de forma exitosa el control y la adquisición de datos de periféricos médicos simulados mediante el mecanismo de **E/S Mapeada en Memoria (MMIO)**. Al declarar las variables de control, estado y datos dentro del segmento `.data`, se emula el comportamiento de un mapa de direcciones de hardware físico, donde escribir en una etiqueta activa un transductor externo y leer de otra devuelve su estado de operación corriente.

- **Caso 1 - Sensor de Luz:** El algoritmo gestiona de manera lógica tres condiciones del hardware: la inicialización del dispositivo activando el registro `LuzControl`, una sincronización síncrona mediante bucles de espera activa para verificar la preparación del transductor (`LuzEstado == 1`), y una rutina dedicada a capturar fallos críticos de hardware (`LuzEstado == -1`). Esta estructura garantiza la robustez del sistema frente a lecturas erróneas de luminosidad.

- **Caso 2 - Tensiómetro Digital:** La subrutina `controlador_tension` demuestra la viabilidad de adquirir múltiples señales físicas de forma concurrente de un solo dispositivo de medición. El código suspende de forma coordinada la línea principal de ejecución del procesador mientras dure la oscilometría neumática (`TensionEstado == 0`). Al concluir, efectúa la lectura consecutiva de dos variables independientes: la presión sistólica (`TensionSistol`) y la diastólica (`TensionDiastol`), cargándolas simultáneamente en los registros de retorno estándar de MIPS.